2 Requirements engineering

Week 2 · Worksheet

Name _____ Date _____

Exercises

Show your work. Each exercise has a hint at the end: try without it first.

1. Problem or requirement? Warm-up · Lesson 1

Label each line as a problem statement (P) or a requirement (R): (a) "Our delivery customers complain that food arrives cold." (b) "The app shows the estimated arrival time, updated every minute." (c) "Gym members say the 6 pm classes are always full." (d) "Members can join a waiting list for a full class."

Hint: A problem describes pain; a requirement describes something the system will do.

2. Find three causes Practice · Lesson 1

A food-delivery company says: "Many customers abandon their order before paying." Write three possible causes and one requirement for each.

Hint: Think about price surprises, effort, and trust.

3. Add the constraints Challenge · Lesson 1

The wish is: "Readers can see everything they have ever borrowed in the library app." Name one legal, one social, and one security constraint, and rewrite the wish as a requirement that respects them.

Hint: Who may see a reading history? Does every reader want one kept at all?

4. Sort the library list Warm-up · Lesson 2

Mark each one F (functional) or NF (non-functional): (a) Librarians can mark a copy as damaged. (b) The catalogue page meets WCAG 2.1 AA accessibility. (c) Readers receive an email when a reserved book is ready. (d) The system runs on the library's existing Windows servers. (e) Catalogue search results appear in under 1 second.

Hint: Is it an action with a visible result, or a quality or limit?

5. Make it measurable Practice · Lesson 2

Rewrite with a measure: (a) "The delivery app should be reliable." (b) "The gym app should be easy to learn." (c) "The system should handle lots of users."

Hint: Availability percentage, time to learn, concurrent users with a response time.

6. From user to system requirement Challenge · Lesson 2

Expand the user requirement "A gym member can book a class" into four precise system requirements.

Hint: Think about who can book, how far ahead, capacity, and what happens when a class is full.

7. List the stakeholders Warm-up · Lesson 3

List at least six stakeholders of a gym membership app and say what each one cares about.

Hint: Include people who use it, people who pay, people who maintain it, and outside bodies.

8. Name the activity Practice · Lesson 3

Which activity is each? (a) Showing a clickable mock-up to desk staff to confirm the lending flow. (b) Checking if an SMS gateway fits the budget. (c) Logging that the hold pickup window changed from 7 to 3 days, approved by the head librarian. (d) Sitting at the lending desk for a morning to watch how loans happen.

Hint: Check with the client, check if possible, record a change, gather facts.

9. Choose the technique Warm-up · Lesson 4

Pick a technique for each: (a) You need the opinion of 3,000 gym members about class times. (b) You must learn the rules for keeping readers' personal data. (c) Couriers cannot describe what slows them down at restaurants. (d) The owner cannot imagine how the new booking screen should look.

Hint: Many people, written rules, invisible habits, need something concrete.

10. Find the hidden policies Practice · Lesson 4

Requirement: "Members who miss classes are penalised." List three business-policy questions the client must answer before anyone codes it.

Hint: What counts as missing? What is the penalty? Are there exceptions?

11. Draw the boundary Challenge · Lesson 4

Describe in words a context diagram for a public library system: the system, at least four external actors, and one data flow each way for two of them. Then write one "in scope" and one "out of scope" statement.

Hint: Actors are outside the system: people, other systems, organisations.

12. Spot the defect Warm-up · Lesson 5

Name the main quality problem: (a) "The app should be user-friendly." (b) "RQ-3: refunds within 7 days. RQ-9: no refunds after 3 days." (c) "The system shows the menu and accepts payment and sends receipts and tracks couriers." (d) "Members can pause their plan." (nothing on how long, how often, or what it costs).

Hint: Ambiguous, inconsistent, not atomic, incomplete.

13. Write a requirement card Practice · Lesson 5

Write a full card (ID, type, priority, description, source, acceptance) for: "gym members get a reminder before a class".

Hint: When exactly? Through which channel? What if the member turned notifications off?

14. Enduring or volatile? Warm-up · Lesson 6

Food-delivery app: (a) Customers order food from restaurants. (b) The delivery fee is waived on orders over $25 this month. (c) Couriers must show a new city-issued badge number from next year. (d) After launch, restaurants ask for a "pause orders" button during rush hour.

Hint: Core of the business, short promotion, new rule from outside, need created by using the system.

15. Impact analysis Practice · Lesson 6

Using the matrix in this lesson, the privacy law changes and now requires a log of every time staff open a reader's reading history. List the rows and items you would review.

Hint: Find the row whose source is the law, then follow its design and test columns.

16. Complete the template Warm-up · Lesson 7

Turn each need into a story: (a) a courier wants to see the pickup address before accepting an order; (b) a gym owner wants a weekly report of cancelled memberships.

Hint: Role, goal, and a benefit that explains why it matters.

17. Which INVEST letter fails? Practice · Lesson 7

(a) "As a member, I want the whole app, so that I can use the gym." (b) "As a member, I want the app to be nice." (c) "As a member, I want to pay for a class, which only works after story 14, which needs story 9." (d) "As the admin, I want the database migrated to version 16."

Hint: Too big, cannot be tested, chained to others, no user value.

18. MoSCoW a first release Challenge · Lesson 7

A library app has six stories for its first release in four weeks: reserve a book, renew a loan, email reminder before a due date, favourite branch, dark mode, e-book lending. Sort them with MoSCoW and justify one decision.

Hint: What makes the release useless if it is missing? What is a big new product on its own?

19. Label the parts Warm-up · Lesson 8

Label each line Given, When, or Then: (a) the order total is $8; (b) the customer taps "Place order"; (c) the app shows "Minimum order is $10"; (d) the restaurant's minimum order is $10.

Hint: Situation before, user action, system response.

20. Write the unhappy path Practice · Lesson 8

Story: "As a member, I want to cancel my class booking in the app, so that someone else can use my spot." The rule is: cancellation is allowed up to 2 hours before the class. Write one happy and one unhappy scenario in Given–When–Then.

Hint: Use concrete times just inside and just outside the limit.

21. Slice an epic Challenge · Lesson 8

Epic: "As a gym owner, I want to manage classes, so that members always see an accurate timetable." Split it into at least four stories using two different slicing patterns, and say which slices you would release first.

Hint: Try operations (create, edit, cancel) and roles (owner, trainer).

22. A full worked example Challenge · Lesson 8

Write story US-02 for a food-delivery app: a customer schedules an order for later. Give it three Given–When–Then criteria: the normal case and two edge cases.

Hint: Edge cases: a time when the restaurant is closed, and a dish that runs out before the scheduled time.

Quick check

Circle one answer.

23. A client writes: "Store each order in PostgreSQL with a UUID key." What is it?

A) A design decision, not a requirement B) A functional requirement C) A business requirement

24. Which statement is a system requirement rather than a user requirement?

A) A returned copy that someone reserved waits 3 days on the hold shelf, then passes to the next reader in the queue.

B) Readers can reserve books online. C) The library wants shorter waits for popular books.

25. A gym app goes live. A week later, the city's consumer-protection office says members must be able to cancel online as easily as they signed up. Which step was weak?

A) Stakeholder identification and elicitation: a regulator was missed

B) Coding: the developers typed too slowly C) Nothing: laws are not requirements

26. Librarians cannot picture how a new self-service loan kiosk should work. Which elicitation technique helps most?

A) A clickable prototype they can try B) A questionnaire with 40 questions

C) Reading the library's tax forms

27. In production, gym members are locked out after 2 wrong PINs, and the owner is angry: she wanted 5. The developer "picked a safe number". What went wrong?

A) A business policy was decided by the programmer instead of being made explicit for the client

B) The PIN screen used the wrong colour C) Lockouts should never exist

28. Which requirement is verifiable?

A) The courier's position on the map refreshes at least every 15 seconds. B) Tracking should feel smooth.

C) The map should be as accurate as possible.

29. After the library introduces self-service kiosks, desk staff ask for a screen to fix loans the kiosks recorded wrongly, because the kiosks changed how lending works. What kind of requirement is this?

A) Consequential (volatile): it exists because the system was introduced

B) Enduring: it was always part of the business

C) Compatibility: it depends on another company's software

30. In MoSCoW, what does "Won't have (this time)" mean?

A) Agreed to be out of this release, but it may come later B) Rejected forever

C) Will be done only if the team works overtime

Answer key

1. (a) P, (b) R, (c) P, (d) R. Notice that (b) and (d) each attack one possible cause of (a) and (c).
2. Example: (1) The delivery fee appears only at the end → "The delivery fee is shown on the restaurant page before items are added." (2) Checkout asks for too much data → "Returning customers pay with a saved card in at most two taps." (3) Customers do not trust the payment page → "Payment uses a certified provider and shows its seal." Any set of distinct causes with matching requirements is fine.
3. Legal: what a person reads is private personal data; only the reader may see it. Social: some readers do not want any record kept, so history must be optional. Security: access requires a verified login. Requirement: "Readers who switch reading history on can see the titles they borrowed in the last 2 years after logging in; with history off, a loan leaves no record once the book is returned."
4. (a) F, (b) NF (external: accessibility standard), (c) F, (d) NF (organisational: platform), (e) NF (performance).
5. (a) "Order placement is available 99.5% of each month, and no more than 1 in 1,000 orders fails." (b) "A first-time member books a class within 2 minutes without help, in 9 of 10 usability tests." (c) "With 2,000 users online at the same time, 95% of pages load in under 2 seconds."
6. Example: (1) Only members with an active plan can book. (2) Classes open for booking 7 days ahead and close 30 minutes before start. (3) A class accepts at most its room capacity; when full, the member is offered the waiting list. (4) A member can hold at most 2 future bookings for the same class type.
7. Members (easy booking), trainers (their class lists), front-desk staff (quick check-in), the owner (renewals and revenue), the payment provider (correct card handling), IT support (easy maintenance), the data-protection authority (privacy of member data). Six or more sensible ones is a pass.
8. (a) Validation. (b) Feasibility. (c) Management. (d) Elicitation (observation).
9. (a) Questionnaire. (b) Document analysis (laws and regulations). (c) Observation (ride along or wait at pickup points). (d) Prototype (sketch or clickable mock-up).
10. Examples: How late can a member cancel before it counts as a no-show? How many no-shows trigger a penalty, and in what period? Is the penalty a fee, a booking ban for N days, or a warning? Are medical excuses accepted? Each answer becomes a configurable value.
11. System in the centre. Actors: reader, librarian, email/SMS provider, regional library network, e-book distributor. Reader → system: reservation request; system → reader: "your book is ready" notice. Regional network → system: request to lend a copy; system → regional network: copy availability. In scope: "catalogue search, reservations, loans, renewals, and reminders". Out of scope: "buying new books and staff payroll".
12. (a) Ambiguous and unverifiable. (b) Inconsistent: the two contradict each other between day 3 and day 7. (c) Several requirements in one: split into four. (d) Incomplete: limits and rules are missing.
13. ID: RQ-21. Type: functional. Priority: Should. Description: 2 hours before a booked class starts, the app sends a push notification with the class name, room, and a cancel link; members who disabled notifications receive nothing. Source: gym owner. Acceptance: booking at 18:00 → notification at 16:00; booking cancelled before 16:00 → no notification; notifications off → no notification.
14. (a) Enduring. (b) Volatile (changing: a business promotion). (c) Volatile (changing: an external rule). (d) Volatile (consequential or emergent: it appeared because the system is now in use).
15. RQ-11 (source: privacy law) changes. Review its design element (ReadingHistory access) and its test T-11a, and add a new requirement for the access log with its own tests. No other row depends on RQ-11, so the impact is contained.
16. (a) "As a courier, I want to see the pickup address before I accept an order, so that I only take orders on my route." (b) "As the gym owner, I want a weekly report of cancelled memberships, so that I can contact those members and find out why they left."

17. (a) Small (and Estimable): it is an epic or bigger. (b) Testable: "nice" cannot be checked. (c) Independent: it depends on a chain of stories. (d) Valuable: it is a technical task with no visible user benefit; attach it to a story it enables.
18. One sensible answer: Must: reserve, renew. Should: email reminder (the library wants books back on time, but staff can phone as a workaround). Could: favourite branch, dark mode. Won't (this time): e-book lending, an epic with licensing and technical questions that does not fit in four weeks.
19. (a) Given, (d) Given (or And), (b) When, (c) Then. In order: Given the restaurant's minimum is $10 And the total is $8, When the customer taps "Place order", Then the app shows the minimum-order message.
20. Happy: Given my yoga class is at 18:00 And it is 15:30, When I cancel my booking, Then it is cancelled And one more spot shows as free. Unhappy: Given my yoga class is at 18:00 And it is 16:30, When I try to cancel, Then I see "Cancellations close 2 hours before class" And my booking stays active.
21. By operation: create a class; edit a class time; cancel a class (booked members are notified); hide a class temporarily. By role: as a trainer, I want to see who booked my class, so that I can prepare equipment. First release: create, edit, and cancel with notification, because without them the timetable cannot be accurate; hiding and the trainer view can wait.
22. US-02: As a customer, I want to schedule an order for a later time, so that dinner arrives when I get home. AC1 (normal): Given the restaurant is open until 22:00, When I schedule my order for 20:30, Then the order is confirmed for 20:30 And I am charged now. AC2 (restaurant closed): Given the restaurant closes at 22:00, When I choose 22:30, Then I see "Choose a time before 21:30" And no order is created. AC3 (dish sold out): Given my scheduled order includes lasagne, When the restaurant marks lasagne as sold out before 20:30, Then I get a notice to replace the dish or cancel And I am refunded if I cancel.
23. **A** — It says how to build something (database and key type), not what the user needs. Unless the client imposes PostgreSQL as an organisational constraint, it belongs to design.
24. **A** — The first gives developers precise detail. The second is a user requirement in plain language, and the third is a business goal.
25. **A** — External bodies are stakeholders and laws produce external non-functional requirements. Missing them early forces an expensive change after release.
26. **A** — People react to something concrete more easily than they describe a need from nothing. A prototype also helps validation later.
27. **A** — The number of attempts is a business policy. Analysis should surface it as a question for the client, and the code should keep it configurable.
28. **A** — Only the first has a measurable target a test can pass or fail. "Smooth" and "as accurate as possible" have no limit.
29. **A** — The need appeared as a consequence of introducing the system itself. It should go through change management like any other change.
30. **A** — It makes scope explicit for now; it is not a permanent rejection. That clarity protects the release.